Phase 5: Iterative Traversals and Level Order Traversal

0. Purpose of This Phase

Phase 5 teaches how to traverse a binary tree **without using recursion**.

In Phase 4, you learned recursive traversals:

- Preorder
- Inorder
- Postorder

Those recursive functions worked because the computer automatically used the **system stack**.

In Phase 5, you learn how to do the same work manually by using:

- A **stack** for depth-first traversals
- A **queue** for level-order traversal

The main beginner idea is:

> Recursion uses a hidden stack.
> Iterative traversal uses a stack that we create and control ourselves.

---

1. Current Progress Analysis

Before Phase 5, you should already understand the following ideas.

Previous idea	Why it matters in Phase 5
Tree	Traversal happens on a tree.
Binary tree	Phase 5 uses left and right child pointers.
Root	Traversal usually starts from the root.
Left child and right child	Traversal rules depend on left and right subtrees.
Leaf node	Leaf nodes stop the movement because they have no children.
Internal node	Internal nodes can lead to more children.
Linked representation	Iterative traversal follows child pointers.

Previous idea	Why it matters in Phase 5
`Node*`	The stack and queue store node addresses.
`nullptr`	We check whether a child exists or not.
Recursive traversal	Iterative traversal replaces recursion with loops.
System stack	Manual stack replaces the system stack.
Queue-based construction	Level order also uses a queue.

So your current position is:

> You can build and recursively traverse a binary tree.
> Now you are learning how to traverse the same tree using loops, stacks, and queues.

---

2. What Phase 5 Covers

Phase 5 covers these main topics:

1. Why iterative traversal needs a stack
2. Depth-first traversal using an explicit stack
3. Iterative preorder traversal
4. Iterative inorder traversal
5. Iterative postorder traversal
6. The signed-address idea from the notes
7. A safer modern C++ postorder method using a visited flag
8. Level-order traversal using a queue
9. Difference between stack and queue
10. Difference between DFS and BFS
11. Dry runs for each traversal
12. Complete C++ program
13. Time and space complexity
14. Common beginner mistakes
15. Practice questions and answers

---

3. Key Idea: Recursion vs Iteration

3.1 Recursive Traversal

In recursive traversal, the function calls itself.

Example:

```
preorder(root->left);
```

When this happens, the computer pauses the current function and stores it in the **system stack**.

So recursion automatically remembers:

- Where it came from
- Which node it was processing
- Which child it still needs to visit

3.2 Iterative Traversal

In iterative traversal, we do not call the function again and again.

Instead, we use:

- `while` loops
- `stack<Node*>`
- `queue<Node*>`

The stack or queue remembers which nodes still need work.

Simple comparison:

Recursive traversal	Iterative traversal
Uses function calls	Uses loops
Uses hidden system stack	Uses explicit stack or queue
Easier to write at first	Gives more manual control
Function remembers return points	Stack remembers return points

Beginner memory trick:

> Recursive = computer manages the stack.
> Iterative = you manage the stack.

---

4. DFS and BFS

There are two major ways to move through a tree.

Traversal family	Full name	Data structure	Meaning
DFS	Depth-First Search	Stack	Go deep before moving across.
BFS	Breadth-First Search	Queue	Visit level by level.

4.1 Depth-First Traversal

Depth-first traversal goes down one side of the tree before moving to another side.

These are depth-first traversals:

- Preorder
- Inorder
- Postorder

They use a **stack** when written iteratively.

4.2 Breadth-First Traversal

Breadth-first traversal visits nodes level by level.

This is breadth-first traversal:

- Level order

It uses a **queue**.

Memory trick:

> Stack = depth-first
> Queue = level-by-level

---

5. Stack and Queue Review

5.1 Stack

A stack follows the rule:

> Last In, First Out

Short form:

> LIFO

Example:

If we push:

```
10, 20, 30
```

Then the first value removed is:

```
30
```

because 30 was inserted last.

In tree traversal, a stack is useful because it helps us return to the most recent unfinished node.

5.2 Queue

A queue follows the rule:

> First In, First Out

Short form:

> FIFO

Example:

If we enqueue:

```
10, 20, 30
```

Then the first value removed is:

```
10
```

because 10 was inserted first.

In tree traversal, a queue is useful for level order because the first node discovered should be processed first.

5.3 Stack vs Queue

Data structure	Rule	Used for	Simple idea
Stack	LIFO	DFS	Go deep and come back.
Queue	FIFO	BFS	Visit in discovery order.

---

6. Node Structure Used in Phase 5

We use this binary tree node:

```
struct Node {
    int data;
    Node* lchild;
    Node* rchild;

    Node(int val) {
        data = val;
        lchild = nullptr;
        rchild = nullptr;
    }
};
```

6.1 Explanation

```
int data;
```

This stores the value inside the node.

```
Node* lchild;
```

This stores the address of the left child.

```
Node* rchild;
```

This stores the address of the right child.

```
lchild = nullptr;
rchild = nullptr;
```

This means the node starts with no children.

6.2 Why We Store `Node*` in Stack and Queue

The stack and queue should store node addresses, not just values.

Correct:

```
stack<Node*> st;
queue<Node*> q;
```

Wrong for traversal control:

```
stack<int> st;
queue<int> q;
```

Why?

Because a value like `20` cannot tell us where its left and right children are.

But a `Node*` points to the full node, so we can do:

```
p->lchild
p->rchild
```

Memory trick:

> Values cannot lead to children.
> Node addresses can lead to children.

---

7. Example Tree Used in This Phase

We will use this tree many times:

```
        10
       /
    20     30
```

```
    / \     /
  40   50   60   70
```

Tree facts:

- `10` is the root.
- `20` is the left child of `10`.
- `30` is the right child of `10`.
- `40` and `50` are children of `20`.
- `60` and `70` are children of `30`.
- `40`, `50`, `60`, and `70` are leaf nodes.
- `10`, `20`, and `30` are internal nodes.

Expected traversal outputs:

Traversal	Output
Iterative preorder	`10 20 40 50 30 60 70`
Iterative inorder	`40 20 50 10 60 30 70`
Iterative postorder	`40 50 20 60 70 30 10`
Level order	`10 20 30 40 50 60 70`

Part A: Iterative Preorder Traversal

8. Preorder Rule

Preorder means:

```
Root, Left, Right
```

Simple memory trick:

> Preorder = root first.

In recursive preorder, the root is printed before going left and right.

In iterative preorder, we do the same thing:

1. Print the node when we first see it.
2. Push the node into the stack.
3. Move left.

4. When left is finished, pop and move right.

---

9. Why Preorder Uses a Stack

When we move left, we are not finished with the current node yet.

Why?

Because after finishing the left subtree, we still need to visit the right subtree.

So we push the current node onto the stack before moving left.

The stack means:

> Come back here later and process the right child.

---

10. Iterative Preorder Algorithm

Algorithm:

1. Create an empty stack.
2. Start from the root.
3. While the current node is not null OR the stack is not empty:
4. If the current node is not null:
    - Print it.
    - Push it onto the stack.
    - Move to its left child.
5. Else:
    - Pop a node from the stack.
    - Move to its right child.

Important loop condition:

```
while (t != nullptr || !st.empty())
```

This means:

> Continue while there is a current node OR saved nodes in the stack.

---

11. Iterative Preorder Code

```
void iterativePreorder(Node* t) {
    stack<Node*> st;

    while (t != nullptr || !st.empty()) {
        if (t != nullptr) {
            cout << t->data << " ";
            st.push(t);
            t = t->lchild;
        } else {
            t = st.top();
            st.pop();
            t = t->rchild;
        }
    }
}
```

---

12. Iterative Preorder Explanation

This line creates the stack:

```
stack<Node*> st;
```

The stack stores addresses of nodes.

This loop controls the traversal:

```
while (t != nullptr || !st.empty())
```

The traversal continues while:

- There is a current node, or
- There are saved nodes in the stack

If `t` is not null:

```
cout << t->data << " ";
st.push(t);
t = t->lchild;
```

This means:

1. Print the current node.
2. Save it in the stack.
3. Move left.

If `t` is null:

```
t = st.top();
st.pop();
t = t->rchild;
```

This means:

1. We reached the end of a left path.
2. Go back to the most recent saved node.
3. Move to its right child.

---

13. Iterative Preorder Dry Run

Tree:

```
        10
       /
     20     30
    / \     /
  40  50   60  70
```

Preorder rule:

```
Root, Left, Right
```

Steps:

Step	Action	Output
1	Visit 10	10
2	Go left to 20	10 20
3	Go left to 40	10 20 40
4	40 has no left child, go back	10 20 40

Step	Action	Output
5	40 has no right child, go back to 20	10 20 40
6	Go right to 50	10 20 40 50
7	50 has no children, go back to 10	10 20 40 50
8	Go right to 30	10 20 40 50 30
9	Go left to 60	10 20 40 50 30 60
10	Go right to 70	10 20 40 50 30 60 70

Final preorder output:

```
10 20 40 50 30 60 70
```

Part B: Iterative Inorder Traversal

14. Inorder Rule

Inorder means:

```
Left, Root, Right
```

Simple memory trick:

> Inorder = root in the middle.

In inorder, we do not print the root immediately.

We first go as far left as possible.

---

15. Main Difference Between Preorder and Inorder

Preorder prints before going left.

Inorder prints after coming back from the left.

Traversal	When do we print?
Preorder	When we first see the node
Inorder	When we pop the node from the stack

In inorder, popping means:

> The left subtree is finished. Now visit the root.

---

16. Iterative Inorder Algorithm

Algorithm:

1. Create an empty stack.
2. Start from the root.
3. While the current node is not null OR the stack is not empty:
4. If the current node is not null:
    - Push it onto the stack.
    - Move to its left child.
5. Else:
    - Pop a node from the stack.
    - Print it.
    - Move to its right child.

---

17. Iterative Inorder Code

```
void iterativeInorder(Node* t) {
    stack<Node*> st;

    while (t != nullptr || !st.empty()) {
        if (t != nullptr) {
            st.push(t);
            t = t->lchild;
        } else {
            t = st.top();
            st.pop();
            cout << t->data << " ";
            t = t->rchild;
        }
```

```
    }
}
```

---

18. Iterative Inorder Explanation

When `t` is not null:

```
st.push(t);
t = t->lchild;
```

This means:

1. Save the current node.
2. Move left.

We do **not** print yet.

Why?

Because inorder says left comes before root.

When `t` is null:

```
t = st.top();
st.pop();
cout << t->data << " ";
t = t->rchild;
```

This means:

1. There is no more left child.
2. Pop the last saved node.
3. Print it.
4. Move right.

---

19. Iterative Inorder Dry Run

Tree:

```
        10
       /
     20     30
    / \     /
  40   50   60   70
```

Inorder rule:

```
Left, Root, Right
```

Steps:

Step	Action	Output
1	Go left from 10 to 20	
2	Go left from 20 to 40	
3	40 has no left child, print 40	40
4	Go back to 20, print 20	40 20
5	Go right to 50, print 50	40 20 50
6	Left subtree of 10 is done, print 10	40 20 50 10
7	Go right to 30	40 20 50 10
8	Go left to 60, print 60	40 20 50 10 60
9	Go back to 30, print 30	40 20 50 10 60 30
10	Go right to 70, print 70	40 20 50 10 60 30 70

Final inorder output:

```
40  20  50  10  60  30  70
```

Part C: Iterative Postorder Traversal

20. Postorder Rule

Postorder means:

```
Left, Right, Root
```

Simple memory trick:

> Postorder = root last.

Postorder is the hardest iterative traversal because we must print the root only after both children are finished.

---

21. Why Iterative Postorder Is Harder

For preorder:

- Print root first.

For inorder:

- Print root after left subtree.

For postorder:

- Print root after left subtree and right subtree.

That means we need to know whether a node is being seen:

- For the first time, or
- After its children are already processed

This is why postorder usually needs an extra marker or flag.

---

22. Signed-Address Idea From the Notes

The notes mention a signed-address trick.

The idea is:

- Push a node address normally when going left.
- Later, push the same address with a negative sign to show that the left side is done.
- When the negative marker is found, print the node.

Simple meaning:

> Positive address = visit later.
> Negative address = children are done, print now.

However, this technique is not safe in modern C++ because pointer addresses should not be converted into negative integers.

So for beginner-friendly modern C++, it is better to use a safer method.

---

23. Safer Postorder Method: Pair With Visited Flag

A safer method uses:

```
stack<pair<Node*, bool>> st;
```

Each stack item stores two things:

```
(node address, visited flag)
```

The visited flag tells us whether this node has already been seen before.

Flag value	Meaning
`false`	First time seeing the node. Its children are not done.
`true`	Children are done. Now print the node.

Memory trick:

> `false` means prepare the children.
> `true` means print the node.

---

24. Iterative Postorder Algorithm Using Visited Flag

Algorithm:

1. If root is null, stop.
2. Create a stack of pairs.
3. Push `(root, false)`.
4. While the stack is not empty:
5. Pop the top pair.
6. If the node is null, skip it.
7. If `visited == true`, print the node.

8. If `visited == false`:
    - Push `(node, true)`.
    - Push `(right child, false)`.
    - Push `(left child, false)`.

Why push root, then right, then left?

Because stack is LIFO.

The last thing pushed is processed first.

So pushing left last makes left processed first.

---

25. Iterative Postorder Code

```
void iterativePostorder(Node* root) {
    if (root == nullptr) return;

    stack<pair<Node*, bool>> st;
    st.push({root, false});

    while (!st.empty()) {
        auto [node, visited] = st.top();
        st.pop();

        if (node == nullptr) continue;

        if (visited) {
            cout << node->data << " ";
        } else {
            st.push({node, true});
            st.push({node->rchild, false});
            st.push({node->lchild, false});
        }
    }
}
```

---

26. Iterative Postorder Explanation

Empty tree check

```
if (root == nullptr) return;
```

If the tree has no root, there is nothing to print.

Create stack

```
stack<pair<Node*, bool>> st;
```

The stack stores:

- A node address
- A boolean flag

Push root as not visited

```
st.push({root, false});
```

This means:

> We have not processed root's children yet.

If visited is true

```
if (visited) {
    cout << node->data << " ";
}
```

This means:

> Left and right children are done. Print the root now.

If visited is false

```
st.push({node, true});
st.push({node->rchild, false});
st.push({node->lchild, false});
```

This prepares the correct postorder sequence:

```
Left, Right, Root
```

Because stack is LIFO, left will be processed first.

---

27. Iterative Postorder Dry Run

Tree:

```
        10
       /
     20     30
    / \     /
  40   50   60   70
```

Postorder rule:

```
Left, Right, Root
```

Expected logic:

1. Finish left subtree of 10.
2. Finish right subtree of 10.
3. Print 10 last.

Left subtree of 10 is rooted at 20:

```
40 50 20
```

Right subtree of 10 is rooted at 30:

```
60 70 30
```

Then root:

```
10
```

Final postorder output:

```
40 50 20 60 70 30 10
```

Part D: Level Order Traversal

28. What Is Level Order Traversal?

Level order means:

```
Visit nodes level by level, from left to right.
```

It is also called breadth-first traversal.

For this tree:

```
        10
       /
     20    30
    / \    /
  40  50  60  70
```

Levels are:

```
Level 1: 10
Level 2: 20 30
Level 3: 40 50 60 70
```

So level order is:

```
10 20 30 40 50 60 70
```

---

29. Why Level Order Uses a Queue

Level order visits nodes in the order they are discovered.

The first node discovered should be processed first.

That is exactly how a queue works.

Queue rule:

```
First In, First Out
```

So for level order:

1. Visit root.
2. Put root into queue.
3. Remove front node.
4. Add its left child.
5. Add its right child.
6. Repeat until queue is empty.

---

30. Level Order Algorithm

Algorithm:

1. If root is null, stop.
2. Create an empty queue.
3. Push root into the queue.
4. While the queue is not empty:
5. Remove the front node.
6. Print it.
7. If it has a left child, push the left child.
8. If it has a right child, push the right child.

---

31. Level Order Code

```
void levelOrder(Node* root) {
    if (root == nullptr) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
        Node* p = q.front();
        q.pop();
```

```
        cout << p->data << " ";

        if (p->lchild != nullptr) q.push(p->lchild);
        if (p->rchild != nullptr) q.push(p->rchild);
    }
}
```

---

32. Level Order Explanation

Empty tree check

```
if (root == nullptr) return;
```

If the tree is empty, stop.

Create queue

```
queue<Node*> q;
```

The queue stores addresses of nodes.

Start with root

```
q.push(root);
```

The root is the first node to process.

Process the queue

```
while (!q.empty())
```

Continue until no nodes are waiting.

Remove front node

```
Node* p = q.front();
q.pop();
```

This gets the next node in level order.

Print it

```
cout << p->data << " ";
```

Add its children

```
if (p->lchild != nullptr) q.push(p->lchild);
if (p->rchild != nullptr) q.push(p->rchild);
```

Add left child first, then right child, because level order moves left to right.

---

33. Level Order Dry Run

Tree:

```
        10
       /
     20     30
    / \     /
  40   50   60   70
```

Initial queue:

```
[10]
```

Dry run:

Step	Remove from queue	Print	Add children	Queue after step
1	10	10	20, 30	[20, 30]
2	20	20	40, 50	[30, 40, 50]
3	30	30	60, 70	[40, 50, 60, 70]
4	40	40	none	[50, 60, 70]
5	50	50	none	[60, 70]
6	60	60	none	[70]

Step	Remove from queue	Print	Add children	Queue after step
7	70	70	none	[]

Final level-order output:

```
10 20 30 40 50 60 70
```

Part E: Complete Phase 5 C++ Program

34. Complete Program

```cpp
#include <iostream>
#include <stack>
#include <queue>
#include <utility>
using namespace std;

struct Node {
    int data;
    Node* lchild;
    Node* rchild;

    Node(int val) {
        data = val;
        lchild = nullptr;
        rchild = nullptr;
    }
};

void iterativePreorder(Node* t) {
    stack<Node*> st;

    while (t != nullptr || !st.empty()) {
        if (t != nullptr) {
            cout << t->data << " ";
            st.push(t);
            t = t->lchild;
        } else {
            t = st.top();
            st.pop();
            t = t->rchild;
        }
```

```
    }
}

void iterativeInorder(Node* t) {
    stack<Node*> st;

    while (t != nullptr || !st.empty()) {
        if (t != nullptr) {
            st.push(t);
            t = t->lchild;
        } else {
            t = st.top();
            st.pop();
            cout << t->data << " ";
            t = t->rchild;
        }
    }
}

void iterativePostorder(Node* root) {
    if (root == nullptr) return;

    stack<pair<Node*, bool>> st;
    st.push({root, false});

    while (!st.empty()) {
        auto [node, visited] = st.top();
        st.pop();

        if (node == nullptr) continue;

        if (visited) {
            cout << node->data << " ";
        } else {
            st.push({node, true});
            st.push({node->rchild, false});
            st.push({node->lchild, false});
        }
    }
}

void levelOrder(Node* root) {
    if (root == nullptr) return;

    queue<Node*> q;
    q.push(root);

    while (!q.empty()) {
```

```
        Node* p = q.front();
        q.pop();

        cout << p->data << " ";

        if (p->lchild != nullptr) q.push(p->lchild);
        if (p->rchild != nullptr) q.push(p->rchild);
    }
}

int main() {
    Node* root = new Node(10);
    root->lchild = new Node(20);
    root->rchild = new Node(30);
    root->lchild->lchild = new Node(40);
    root->lchild->rchild = new Node(50);
    root->rchild->lchild = new Node(60);
    root->rchild->rchild = new Node(70);

    cout << "Iterative Preorder: ";
    iterativePreorder(root);
    cout << endl;

    cout << "Iterative Inorder: ";
    iterativeInorder(root);
    cout << endl;

    cout << "Iterative Postorder: ";
    iterativePostorder(root);
    cout << endl;

    cout << "Level Order: ";
    levelOrder(root);
    cout << endl;

    return 0;
}
```

---

35. Tree Created by the Program

The program creates this tree:

```
        10
       /
```

```
     20    30
    / \    /
  40  50  60  70
```

This line creates the root:

```
Node* root = new Node(10);
```

These lines create the children of `10`:

```
root->lchild = new Node(20);
root->rchild = new Node(30);
```

These lines create the children of `20`:

```
root->lchild->lchild = new Node(40);
root->lchild->rchild = new Node(50);
```

These lines create the children of `30`:

```
root->rchild->lchild = new Node(60);
root->rchild->rchild = new Node(70);
```

---

36. Expected Output

```
Iterative Preorder: 10 20 40 50 30 60 70
Iterative Inorder: 40 20 50 10 60 30 70
Iterative Postorder: 40 50 20 60 70 30 10
Level Order: 10 20 30 40 50 60 70
```

Part F: Time and Space Complexity

37. Time Complexity

All four traversals visit every node once.

So for a tree with `N` nodes:

```
Time complexity = O(N)
```

This applies to:

- Iterative preorder
- Iterative inorder
- Iterative postorder
- Level order

Why?

Because if there are 7 nodes, we visit 7 nodes.

If there are 100 nodes, we visit 100 nodes.

So the time grows with the number of nodes.

---

38. Space Complexity for DFS Traversals

Preorder, inorder, and postorder use a stack.

The stack stores nodes that are waiting to be processed.

For a balanced tree, the stack is usually related to the height of the tree.

So extra space is usually:

```
O(H)
```

Here:

```
H = height of the tree
```

In the worst case, if the tree is skewed, height can be close to `N`.

So worst-case stack space can become:

```
O(N)
```

39. Space Complexity for Level Order

Level order uses a queue.

The queue may store many nodes from one level at the same time.

In the worst case, this can be:

```
O(N)
```

So:

```
Level order extra space = O(N) worst case
```

40. Complexity Summary Table

Traversal	Data structure	Time	Extra space
Iterative preorder	Stack	`O(N)`	`O(H)` usually, `O(N)` worst case
Iterative inorder	Stack	`O(N)`	`O(H)` usually, `O(N)` worst case
Iterative postorder	Stack	`O(N)`	`O(H)` usually, `O(N)` worst case
Level order	Queue	`O(N)`	`O(N)` worst case

Part G: Common Beginner Mistakes

41. Mistake 1: Printing Inorder Too Early

Wrong idea:

> Print the node as soon as you see it.

That is preorder, not inorder.

Correct idea:

> In inorder, print after popping from the stack.

Why?

Because the left subtree must be completed first.

---

42. Mistake 2: Using a Queue for DFS

Wrong idea:

> Use a queue for preorder, inorder, or postorder.

Correct idea:

> Use a stack for DFS traversals.

Preorder, inorder, and postorder are depth-first.

Depth-first uses a stack.

---

43. Mistake 3: Using a Stack for Level Order

Wrong idea:

> Use a stack for level order.

Correct idea:

> Use a queue for level order.

Level order must process nodes in the order they are discovered.

That is FIFO behavior.

---

44. Mistake 4: Stopping When `t == nullptr`

Wrong loop:

```
while (t != nullptr)
```

Problem:

Even if `t` becomes null, there may still be saved nodes in the stack.

Correct loop:

```
while (t != nullptr || !st.empty())
```

This means:

> Continue while there is a current node or saved work in the stack.

---

45. Mistake 5: Forgetting to Check `nullptr`

Wrong idea:

```
cout << root->data;
```

without checking whether `root` is null.

Correct idea:

```
if (root == nullptr) return;
```

or in loops:

```
if (node == nullptr) continue;
```

A null pointer means no node exists there.

---

46. Mistake 6: Confusing Stack and Queue Behavior

Stack:

```
Last In, First Out
```

Queue:

```
First In, First Out
```

If you confuse these, your traversal order will be wrong.

---

47. Mistake 7: Unsafe Postorder Pointer Casting

The signed-address trick is useful for understanding old lecture code, but it is not ideal in modern C++.

Safer method:

```
stack<pair<Node*, bool>> st;
```

This avoids converting pointer addresses into negative values.

---

48. Mistake 8: Pushing Children in the Wrong Order for Postorder

In the visited-flag method, we push:

```
st.push({node, true});
st.push({node->rchild, false});
st.push({node->lchild, false});
```

Why this order?

Because stack is LIFO.

The left child is pushed last, so it is processed first.

This helps produce:

```
Left, Right, Root
```

Part H: Key Vocabulary Table

49. Phase 5 Vocabulary

Term	Simple meaning
Iterative traversal	Traversal using loops instead of recursion
Recursive traversal	Traversal using function calls
System stack	Hidden stack used by recursive calls
Explicit stack	Stack created by the programmer
Stack	LIFO data structure
LIFO	Last In, First Out
Queue	FIFO data structure
FIFO	First In, First Out
DFS	Depth-first search; goes deep before moving across
BFS	Breadth-first search; visits level by level
Preorder	Root, Left, Right
Inorder	Left, Root, Right
Postorder	Left, Right, Root
Level order	Top to bottom, left to right
`Node*`	Pointer to a node
`nullptr`	Means no node exists there
`stack<Node*>`	Stack that stores node addresses
`queue<Node*>`	Queue that stores node addresses
`pair<Node*, bool>`	Stores a node and a visited flag
Visited flag	Tells whether a node's children are already processed

---

Part I: Phase 5 Summary

50. Main Summary

By the end of Phase 5, you should know:

- Recursive traversal uses the hidden system stack.
- Iterative traversal uses loops.
- Iterative DFS uses an explicit stack.
- Level order uses a queue.
- Preorder means Root, Left, Right.
- Inorder means Left, Root, Right.
- Postorder means Left, Right, Root.
- Level order means top to bottom, left to right.
- Iterative preorder prints when first seeing the node.
- Iterative inorder prints after popping the node.
- Iterative postorder prints after both children are done.
- A visited flag is a safe way to write iterative postorder.
- Stack follows LIFO.
- Queue follows FIFO.
- Stack is used for depth-first traversal.
- Queue is used for breadth-first traversal.
- All traversals take `O(N)` time.
- DFS stack space is usually `O(H)`.
- Level order queue space is `O(N)` in the worst case.

---

51. Final Memory Tricks

Traversal rules

Traversal	Rule	Trick
Preorder	Root, Left, Right	Root first
Inorder	Left, Root, Right	Root middle
Postorder	Left, Right, Root	Root last
Level order	Level by level	Read like a book

Data structure tricks

Data structure	Trick
Stack	Go deep and come back

Data structure	Trick
Queue	Process in discovery order

Code tricks

Traversal	Important code idea
Preorder	Print before going left
Inorder	Print after popping
Postorder	Use visited flag
Level order	Pop front, print, push children

Biggest Phase 5 idea

Recursion used the system stack automatically.
Iterative traversal uses your own stack manually.
Level order uses a queue because it moves level by level.

Part J: Practice Questions

Use this tree:

```
        10
       /
    20      30
   /  \     /
40   50   60   70
```

Answer these questions:

1. What is the preorder traversal?
2. What is the inorder traversal?
3. What is the postorder traversal?
4. What is the level-order traversal?
5. Which traversals use a stack?
6. Which traversal uses a queue?
7. What does LIFO mean?
8. What does FIFO mean?
9. Why does iterative preorder print before moving left?
10. Why does iterative inorder print after popping?
11. Why is iterative postorder harder than preorder and inorder?

12. What does the visited flag mean in iterative postorder?
13. Why do we push right before left in the visited-flag postorder method?
14. What is the time complexity of iterative preorder?
15. What is the time complexity of level order?
16. What is the extra space of DFS traversal?
17. What is the worst-case extra space of level order?
18. Why should the stack store `Node*` instead of `int`?
19. What happens if the loop condition is only `while (t != nullptr)`?
20. Why is `stack<pair<Node*, bool>>` safer than signed-address pointer tricks?

Part K: Practice Answers

Answer 1

Preorder means:

```
Root, Left, Right
```

So the preorder traversal is:

```
10 20 40 50 30 60 70
```

Answer 2

Inorder means:

```
Left, Root, Right
```

So the inorder traversal is:

```
40 20 50 10 60 30 70
```

Answer 3

Postorder means:

```
Left, Right, Root
```

So the postorder traversal is:

```
40 50 20 60 70 30 10
```

Answer 4

Level order means level by level:

```
10 20 30 40 50 60 70
```

Answer 5

These traversals use a stack:

- Iterative preorder
- Iterative inorder
- Iterative postorder

They are depth-first traversals.

Answer 6

Level order uses a queue.

It is breadth-first traversal.

Answer 7

LIFO means:

```
Last In, First Out
```

The last item inserted is removed first.

Answer 8

FIFO means:

```
First In, First Out
```

The first item inserted is removed first.

Answer 9

Iterative preorder prints before moving left because preorder means root first.

Rule:

```
Root, Left, Right
```

Answer 10

Iterative inorder prints after popping because popping means the left subtree is finished.

Then it is time to print the root.

Rule:

```
Left, Root, Right
```

Answer 11

Iterative postorder is harder because the root must be printed after both children.

Rule:

```
Left, Right, Root
```

So we need to know whether the children have already been processed.

Answer 12

The visited flag tells whether the node has already been seen before.

- `false` means first time seeing the node.
- `true` means children are done, so print the node.

Answer 13

We push right before left because a stack is LIFO.

If left is pushed last, left is processed first.

This gives the correct postorder direction:

```
Left, Right, Root
```

Answer 14

The time complexity of iterative preorder is:

```
O(N)
```

because every node is visited once.

Answer 15

The time complexity of level order is:

```
O(N)
```

because every node is visited once.

Answer 16

DFS traversal uses stack space usually based on tree height:

```
O(H)
```

In the worst case of a skewed tree, this can become:

```
O(N)
```

Answer 17

The worst-case extra space of level order is:

```
O(N)
```

because the queue may store many nodes from one level.

Answer 18

The stack should store `Node*` instead of `int` because traversal needs access to child pointers.

A value like `20` cannot lead to its children.

A node pointer can:

```
p->lchild
p->rchild
```

Answer 19

If the loop condition is only:

```
while (t != nullptr)
```

then the traversal may stop too early.

Even when `t` becomes null, the stack may still contain nodes waiting for their right subtrees.

Correct condition:

```
while (t != nullptr || !st.empty())
```

Answer 20

`stack<pair<Node*, bool>>` is safer because it does not convert pointer addresses into negative integers.

It stores a clear visited flag instead.

This is easier to understand and safer in modern C++.

---

52. End of Phase 5

You now understand how to traverse a binary tree without recursion.

The most important idea is:

> Recursive traversal hides the stack.
> Iterative traversal makes the stack visible.

After Phase 5, you are ready for Phase 6, where traversal logic is used to solve tree problems such as:

- Counting nodes
- Counting degree-2 nodes
- Finding the sum of all nodes
- Calculating height